

Unidad 2

Datos básicos para algoritmos y programas

Tabla de contenido

1. Sistema de numeración binaria	3
2. Sistema de numeración hexadecimal	5
3. Álgebra de Boole o álgebra booleana	9
2.1 Leyes fundamentales del álgebra de Boole	9
.....	9
2.2 Tablas de verdad de los operadores booleanos.....	10
4. Operaciones booleanas	11
5. Variables, constantes y cómo usarlas	12
5.1. Variables y constantes.....	12
5.2. Tipos de datos simples.....	14
5.3. Nombres.....	14
5.4. Literales	15
5.5. Operadores.....	16
6. Expresiones	18

1. Sistema de numeración binaria



Representación de la información

La información se procesa y almacena en un ordenador como una serie de 1 y 0. Funciona en sistema numérico binario.

El **sistema numérico binario** es una forma de escribir números usando solo dos dígitos: 0 y 1. Estos se utilizan en los ordenadores como una serie de interruptores de "apagado" y "encendido". En binario, el valor posicional de cada dígito es el doble que el del siguiente dígito a la derecha (ya que cada dígito contiene dos valores). En decimal, el sistema que normalmente usan los humanos, cada dígito contiene diez valores, y el valor posicional aumenta en una potencia de diez (lugar de uno, diez, cien, etc.). El valor posicional del dígito más a la derecha en ambos casos es 1.

Ejemplo: 10110011

El valor posicional del último 1 (posición más a la derecha) es 1.

El valor posicional del 1 anterior es 2.

El valor posicional del 0 anterior es 4.

El valor posicional del 0 anterior es 8.

El valor posicional del 1 anterior es 16.

El valor posicional del 1 anterior es 32.

El valor posicional del 0 anterior es 64.

El valor posicional del 1 anterior es 128.

Sumando todos los valores posicionales que tienen 1s, sería $1+2+16+32+128 = 179$. Para mayor comodidad, los dígitos binarios (bits, para abreviar) generalmente se agrupan en dos grupos de 4 bits. Son 8 bits, o un [byte](#), y se escribe utilizando el [sistema de numeración hexadecimal](#). Esto se mostraría como 1011 0011 = B3.

Ejemplos:

Decimal	Binario	Explicación
0	0	$0+0+0+0+0$
1	1	$0+0+0+0+1$
2	10	$0+0+0+2+0$
3	11	$0+0+0+2+1$
4	100	$0+0+4+0+0$
5	101	$0+0+4+0+1$
6	110	$0+0+4+2+0$
7	111	$0+0+4+2+1$
8	1000	$0+8+0+0+0$

9	1001	0+8+0+0+1
10	1010	0+8+0+2+0
11	1011	0+8+0+2+1
12	1100	0+8+4+0+0
13	1101	0+8+4+0+1
14	1110	0+8+4+2+0
15	1111	0+8+4+2+1
16	10000	16+0+0+0+0
17	10001	16+0+0+0+1
18	10010	16+0+0+2+0
19	10011	16+0+0+2+1
20	10100	16+0+4+0+0
21	10101	16+0+4+0+1
22	10110	16+0+4+2+0
23	10111	16+0+4+2+1
24	11000	16+8+0+0+0
25	11001	16+8+0+0+1
26	11010	16+8+0+2+0
27	11011	16+8+0+2+1
28	11100	16+8+4+0+0
29	11101	16+8+4+0+1
30	11110	16+8+4+2+0
31	11111	16+8+4+2+1

El valor máximo es $2^{\text{num dígitos}} - 1$. Como puedes ver en el ejemplo anterior, $2^5 - 1 = 31$

Para representar un número negativo, se debe usar uno de los dígitos para almacenar el signo (0 para números positivos y 1 para números negativos), por lo que el rango es de: $-2^{\text{numdigits}-1}$ a $2^{\text{numdigits}-1} - 1$. En nuestro ejemplo sería $-2^{5-1} = -15$ y $2^{5-1} - 1 = 15$

Aunque este sistema para almacenar números negativos no es muy utilizado. En cambio, es más común usar el complemento a 2.

El complemento a 2 es una forma más difícil de almacenar negativos. Hay tres pasos para ello:

1. Encuentra el número binario positivo (por ejemplo, $8_{\text{base 10}} = 0000\ 1000_{\text{base 2}}$).
2. Cambia los 1 por 0 y los 0 por 1 (por ejemplo, $0000\ 1000_{\text{base 2}}$ se convierte en $1111\ 0111$ base 2). Esto se conoce como "voltear los bits", o aplicar un NOT lógico a la

representación original en base 2.

3. Suma 1 (por ejemplo, $1111\ 0111_{\text{base } 2} + 1_{\text{base } 2} = 1111\ 1000_{\text{base } 2}$).

Este método es mejor porque:

- Es como el signo y la magnitud; Un número negativo comienza con un 1 y un número positivo comienza con un 0.
- La aritmética binaria funcionará.
- Solo hay un valor para 0 ($0000\ 0000_{\text{base } 2}$).

2. Sistema de numeración hexadecimal

La principal desventaja de los números binarios es que el equivalente de cadena binaria de un número decimal grande en base 10 puede ser bastante largo.

Cuando se trabaja con grandes sistemas digitales, como los ordenadores, es común encontrar números binarios que constan de 8, 16 e incluso 32 dígitos, lo que dificulta tanto la lectura como la escritura sin producir errores, especialmente cuando se trabaja con muchos números binarios de 16 o 32 bits.

Una forma común de superar este problema es organizar los números binarios en grupos o conjuntos de cuatro bits (4 bits). Estos grupos de 4 bits utilizan otro tipo de sistema de numeración también comúnmente utilizado en sistemas informáticos y digitales llamado **números hexadecimales**.

El sistema de numeración "Hexadecimal" o simplemente "Hexadecimal" utiliza el **sistema de Base 16** y es una opción popular para representar valores binarios largos porque su formato es bastante compacto y mucho más fácil de entender en comparación con las cadenas binarias largas de 1 y 0.

Al ser un sistema de base 16, el sistema de numeración hexadecimal utiliza 16 dígitos diferentes con una combinación de números del 0 al 15. En otras palabras, hay 16 símbolos de dígitos posibles.

Sin embargo, existe un problema potencial con el uso de este método de notación de dígitos causado por el hecho de que los números decimales de 10, 11, 12, 13, 14 y 15 normalmente se escriben usando dos símbolos adyacentes. Por ejemplo, si escribimos 10 en hexadecimal, ¿nos referimos al número decimal diez, o al número binario de dos (1 + 0)? Para solucionar este complicado problema, los números hexadecimales que identifican los valores de diez, once, . . . , quince se reemplazan con letras mayúsculas de A, B, C, D, E y F respectivamente.

Luego, en el **sistema de numeración hexadecimal**, usamos los números del 0 al 9 y las letras mayúsculas de la A a la F para representar su equivalente en número binario o decimal, comenzando con el dígito menos significativo en el lado derecho.

Como acabamos de decir, las cadenas binarias pueden ser bastante largas y difíciles de leer, pero podemos hacer la vida más fácil dividiendo estos grandes números binarios en grupos pares para que sean mucho más fáciles de escribir y entender. Por ejemplo, el

siguiente grupo de dígitos binarios 1101 0101 1100 1111⁽²⁾ son mucho más fáciles de leer y entender que 1101010111001111⁽²⁾ cuando están todos agrupados.

En el uso cotidiano del sistema de numeración decimal, usamos grupos de tres dígitos o 000 desde el lado derecho para hacer que un número muy grande, como un millón o un billón, sea más fácil de entender para nosotros y lo mismo ocurre en los sistemas digitales.

Los números hexadecimales son un sistema más complejo que el uso de binarios o decimales y se utilizan principalmente cuando se trata de ordenadores y direcciones de memoria. Al dividir un número binario en grupos de 4 bits, cada grupo o conjunto de 4 dígitos ahora puede tener un valor posible entre "0000" (0) y "1111" ($8 + 4 + 2 + 1 = 15$) dando un total de **16** combinaciones de números diferentes del 0 al 15. No olvides que el "0" también es un dígito válido.

Un grupo de dígitos de 4 bits se llama "nibble" y como también se requieren 4 bits para producir un número hexadecimal, un dígito hexadecimal también se puede considerar como un nibble, o medio byte. A continuación, se requieren dos números hexadecimales para producir un byte completo que oscile entre 00 y FF.

Además, dado que 16 en el sistema decimal es la cuarta potencia de 2 (o 2^4), hay una relación directa entre los números 2 y 16, por lo que un dígito hexadecimal tiene un valor igual a cuatro dígitos binarios.

Debido a esta relación, cuatro dígitos en un número binario se pueden representar con un solo dígito hexadecimal. Esto hace que la conversión entre números binarios y hexadecimales sea muy fácil, y el hexadecimal se puede usar para escribir números binarios grandes con muchos menos dígitos.

Los números del 0 al 9 todavía se usan como en el sistema decimal original, pero los números del 10 al 15 ahora están representados por letras mayúsculas del alfabeto de la A a la F inclusive y la relación entre decimal, binario y hexadecimal se da a continuación.

Números hexadecimales

Número decimal	Número binario de 4 bits	Número hexadecimal
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	8
9	1001	9
10	1010	A
11	1011	B
12	1100	C
13	1101	D
14	1110	E
15	1111	F
16	0001 0000	10 (1+0)
17	0001 0001	11 (1+1)
Continuando hacia arriba en grupos de cuatro		

Usando el número binario original de arriba $1101\ 0101\ 1100\ 1111_2$, esto ahora se puede convertir en un número hexadecimal equivalente de D5CF que es mucho más fácil de leer y entender que una larga fila de 1 y 0 que teníamos antes.

Por lo tanto, al usar la notación hexadecimal, los números digitales se pueden escribir con menos dígitos y con mucha menos probabilidad de que ocurra un error. Del mismo modo, convertir números basados en hexadecimal de nuevo en binario es simplemente la operación inversa.

Las características principales de un **sistema de numeración hexadecimal** es que hay 16 dígitos de conteo distintos del 0 al F, y cada dígito tiene un peso o valor de 16 a partir del bit menos significativo (LSB). Para distinguir los números hexadecimales de los números ordinarios, se utiliza un prefijo de "#", (Hash) antes del valor real del número hexadecimal, #D5CF.

Como la base de un sistema hexadecimal es 16, que también representa el número de símbolos individuales utilizados en el sistema, el subíndice 16 se utiliza para identificar un número expresado en hexadecimal. Por ejemplo, el número hexadecimal anterior se expresa como: $D5CF_{(16)}$



NOTA: Aprenderás más sobre esto en el módulo "Sistemas Informáticos"

3. Álgebra de Boole o álgebra booleana

El álgebra de Boole, o el álgebra booleana, fue ideada por el matemático inglés George Boole (1815-1864) y representa la primera aplicación exitosa de los métodos algebraicos a la lógica.

Un álgebra booleana es un conjunto con solo dos elementos: 1 y 0 (o Verdadero y Falso, T y F, Activado y Desactivado)

Y 3 operaciones: Y (AND), O (OR), NO (NOT)

A veces, el operador AND se escribe como la multiplicación $\cdot$, la operación OR se escribe como $+$ y NOT como $\overline{}$ superior

En Java esos operadores son &&, || y !

2.1 Leyes fundamentales del álgebra de Boole

OR

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \overline{A} = 1$$

AND

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \overline{A} = 0$$

NOT

$$\overline{\overline{0}} = 1$$

$$\overline{\overline{1}} = 0$$

Propiedad conmutativa

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

Propiedad asociativa

$$(A + B) + C = A + (B + C)$$

$$(A \cdot B) \cdot C = A \cdot (B \cdot C)$$

Propiedad distributiva

$$A \cdot (B + C) = (A \cdot B) + (A \cdot C)$$

$$A + (B \cdot C) = (A + B) \cdot (A + C)$$

Leyes de Morgan

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

$$\overline{A + B} = \overline{A} \cdot \overline{B}$$

Propiedades útiles (No aprendas esto)

$$A + (A \cdot B) = A$$

$$A \cdot (A + B) = A$$

$$A + (\overline{A} \cdot B) = A + B$$

$$(A + B) \cdot (A + \overline{B}) = A$$

$$(A + B) \cdot (A + C) = A + (B \cdot C)$$

$$A + B + (A \cdot \overline{B}) = A + B$$

$$(A \cdot B) + (B \cdot C) + (\overline{B} \cdot C) = (A \cdot B) + \overline{B} \cdot C$$

$$(A \cdot B) + (A \cdot C) + (\overline{B} \cdot C) = (A \cdot B) + \overline{B} \cdot C$$

2.2 Tablas de verdad de los operadores booleanos

A	!A
T	F
F	T

A	B	A & B
F	F	F
F	T	F
T	F	F
T	T	T

Un	B	Un B
F	F	F
F	T	T
T	F	T
T	T	T

4. Operaciones booleanas



Un **valor booleano** solo puede tener dos estados posibles:

- verdadero
- falso

o sí/no o 1/0

Igualdad y operadores relacionales en Java

== Igual a

!= No es igual a

> Mayor que

>= Mayor o igual que

< Menor que

<= Menor o igual que

La igualdad y los operadores relacionales generan un resultado booleano.

Por ejemplo: $3 > 4$ ---> falso

$3 == 3$ ---> verdadero

Las operaciones booleanas se realizan mediante los operadores AND y OR y valores booleanos.

Hay más operadores booleanos pero solo necesitaremos 3 de ellos:

Operador	Símbolo	Acción
Operador de complemento lógico	!	Invierte el valor de un valor booleano
Condicional-Y	&&	realiza un AND lógico
Condicional-OR		realiza un OR lógico

Ejemplos:

Expresión	Resultado
9 == (3 * 3)	verdadero
3 != 2	verdadero
(9 == (3*3)) && (3 !=2)	verdadero
(3>2) && ('b'<'a')	falso (porque 'b'>'a')
(3>2) ('b'<'a')	verdadero
! ('a'<'b')	falso
(5>1) && ! ('b'<'a')	verdadero

Los paréntesis se utilizan para cambiar el orden de prioridad en las operaciones. Cuando hay ambigüedad o duda, entonces es mejor usarlos.

5. Variables, constantes y cómo usarlas

5.1. Variables y constantes

Las variables y constantes son ubicaciones de memoria que contienen valores a los que se puede acceder desde un programa mediante un nombre o identificador.

- **Las variables** pueden contener tipos de datos simples (reales, enteros, caracteres, booleanos, etc.) o pueden contener estructuras de datos más complejas (vectores, matrices, registros, etc.).

- **Las constantes** son variables que no pueden cambiar su valor. Su valor se asigna en el momento en que se declaran.



La declaración de los datos (variables y constantes) se realiza para indicar el nombre de cada variable (también conocida como identificador) y el tipo de datos que almacenarán.

Las variables se declaran utilizando el tipo de datos y, a continuación, el identificador de la variable:

<tipo de datos> <identificador>;

También se pueden inicializar:

<tipo de datos> <identificador> = <valor>;

Ejemplos:

```
age int = 35; // Variable inicializada
int days;
float temperature;
boolean firstTime;
String name;
```

Para declarar una **constante** usamos la palabra reservada final:

```
final <tipo de datos> <identificador> = <valor>;
```

Ejemplos:

```
final float PI=3.14;
final int DAYS_YEAR = 365;
float MAX_TEMPERATURE = 180.55;
```

5.2. Tipos de datos simples.

En Java hay 8 tipos de datos simples diferentes

Tipo simple	Qué almacena	Rango
byte	enteros de 8 bits	De -128 a 127
short	enteros de 16 bits	De -32.768 a 32.767
int	enteros de 32 bits	De -2.147.483.648 a 2.147.483.647
long	enteros de 64 bits	-9.223.372.036.854.775.808 al 9.223.372.036.854.775.807
float	Reales en coma flotante de 32 bits	6 dígitos significativos (10^{-46} a 10^{38})
double	Reales en coma flotante de 64 bits	15 dígitos significativos (10^{-324} a 10^{308})
char	Carácter Unicode de 16 bits	
boolean	Variable booleana	true / false

En este tema solo usaremos esta sintaxis básica para declarar tipos simples. Detallaremos más en los próximos capítulos. Aunque con el propósito de estudiar algoritmos también usaremos el tipo **String**.

5.3. Nombres.

"Pero Elodin el Grande conocía los nombres de todas las cosas, y por lo tanto todas las cosas estaban a su disposición".

Patrick Rothfuss - El nombre del viento

"Los nombres son la forma del mundo, y un hombre que puede pronunciarlos está en el camino al poder"

Patrick Rothfuss - El nombre del viento

- "Why do you call this Belows?" I asked Auri.
- "That's its name," she said easily. The wind made her fine hair stream out behind her like a gauzy pennant. "You call things by their names. That's what names are"

Patrick Rothfuss - El nombre del viento

Cada lenguaje de programación tiene su propio conjunto de reglas y convenciones para los tipos de nombres que puede usar, y el lenguaje de programación Java no es diferente.

Las reglas y convenciones para asignar nombres a las variables se pueden resumir de la siguiente manera:

- Los nombres de las variables distinguen entre mayúsculas y minúsculas. El nombre de una variable puede ser cualquier identificador legal: una secuencia ilimitada de letras y dígitos Unicode, que comienza con una letra, el signo de dólar "\$" o el carácter de subrayado "_". La convención, sin embargo, es comenzar siempre los nombres de las variables con una letra, no con "\$" o "_". Además, el carácter del signo de dólar, por convención, nunca se usa en absoluto. Es posible que encuentres algunas situaciones en las que los nombres generados automáticamente contengan el signo de dólar, pero los nombres de las variables siempre deben evitar usarlo. Existe una convención similar para el carácter de subrayado; Si bien es técnicamente legal comenzar el nombre de la variable con "_", se desaconseja esta práctica. No se permiten espacios en blanco.
- Los caracteres siguientes pueden ser letras, dígitos, signos de dólar o caracteres de subrayado. Las convenciones (y el sentido común) también se aplican a esta regla. Al elegir un nombre para las variables, utilice palabras completas en lugar de abreviaturas crípticas. Si lo haces, tu código será más fácil de leer y entender. En muchos casos, también hará que tu código se documente por sí mismo; Los campos denominados speed, cadence y gear, por ejemplo, son mucho más intuitivos que las versiones abreviadas, como S, C y G. También ten en cuenta que el nombre que elijas no debe ser una [palabra clave o una palabra reservada](#).
- Si el nombre que elijas consta de una sola palabra, deletrea esa palabra en minúsculas. Si consta de más de una palabra, escribe en mayúscula la primera letra de cada palabra subsiguiente. Los nombres gearRatio y currentGear son excelentes ejemplos de esta convención. Si la variable almacena un valor constante, como el entero final estático NUM_GEAR = 6, la convención cambia ligeramente, poniendo en mayúscula cada letra y separando las palabras subsiguientes con el carácter de subrayado. Por convención, el carácter de subrayado nunca se utiliza en ningún otro lugar.

5.4. Literales

Un *literal* es la representación del código fuente de un valor fijo; los literales se representan directamente en el código sin necesidad de cálculo. Como se muestra a continuación, es posible asignar un literal a una variable de un tipo primitivo:

```
boolean result = true;
char capitalC = 'C';
byte b = 100;
short s = 10000;
int i = 100000;
// The number 26, in decimal
int decVal = 26;
// The number 26, in hexadecimal
int hexVal = 0x1a;
double d1 = 123.4;
```

```
// same value as d1, but in scientific notation
double d2 = 1.234e2;
float f1 = 123.4f;
```

5.5. Operadores

La siguiente referencia rápida resume los operadores compatibles con el lenguaje de programación Java.

Operador de asignación simple

= Operador de asignación simple
Es el operador más importante, se utiliza para asignar un valor o una expresión a una variable o constante.

Operadores aritméticos

- + Suma (también utilizado para la concatenación de cadenas)
- Resta
- * Multiplicación
- / División
- % Resto de la división entera

Operadores unarios

- + Operador unario positivo; Indica valor positivo (los números son positivo sin esto, sin embargo)
- Operador unario negativo; Niega Una expresión aritmética
- ++ Incremento; Incrementa el valor en 1
- Decremento; Reduce el valor en 1
- ! Complemento lógico; Invierte el valor de un booleano

Operadores relacionales. Devuelve un valor booleano

- == Igual a
- != No es igual a
- > Mayor que
- >= Mayor o igual que
- < Menor de
- <= Menor o igual que

Operadores condicionales o booleanos. Operan sobre valores booleanos y devuelven un valor booleano

&& Y-Condional (AND)

|| O-Condicional (OR)

Operador de comparación de tipos

instanceof Compara un objeto con un tipo especificado

Operadores de desplazamiento bit a bit y bit

~ Complemento bit a bit unario
 << Desplazamiento a la izquierda con signo
 >> Desplazamiento a la derecha con signo
 >>> Desplazamiento a la derecha sin signo
 & AND bit a bit
 ^ OR exclusivo bit a bit
 | OR inclusivo bit a bit

Precedencia del operador

Operadores	Precedencia
Postfix	expr++ expr--
unario	++expr --expr +expr -expr ~ !
multiplicativo	* / %
aditivo	+ -
desplazamiento	<< >> >>>
relacional	< > <= >= instanciade
igualdad	== !=
bit a bit Y	&
O exclusivo bit a bit	^
O inclusivo bit a bit	
Y lógico	&&
O lógico	
ternario	? :
asignación	= += -= *= /= %= &= ^= = <<= >>= >>>=

6. Expresiones



Una expresión es una construcción formada por variables, literales, operadores e invocaciones de funciones, que se construyen de acuerdo con la sintaxis del lenguaje, que se evalúa a un único valor. Ya has visto ejemplos de expresiones, ilustradas en **negrita** a continuación:

Nota:

1. Un literal es una expresión. Ejemplos: "Antonio", 1.23, ...
2. Una constante o una variable es una expresión: Ejemplos: PI, x, name, ...
3. Una función es una expresión: Ejemplos: sin(x), cos(x), ...
4. Una combinación de literales, constantes, variables, funciones y operadores que siguen las reglas sintácticas del lenguaje de programación. Por ejemplo: $\cos(\pi * x) + 1.25 > \sin(y)$

Ejemplos en **negrita**:

```
int numElements = 0;
Nombre de la cadena = "Antonio";
System.out.println("Good morning" + name);

resultado int = 1 + 2;
if (value1 == value2)
    System.out.println("value1 == value2");
```